

252-0027

Introduction to Programming Exercises

Week 3: Introduction to Java

Jan Leonard

**Department of Computer Science
ETH Zurich**

Organization

My name: *Jan Leonard*

For questions: *leonardj@student.ethz.ch*

Please put «[EProg26]» in the subject

New exercises: **Tuesday evening** (normally)

Submit exercises by **Tuesday evening (23:59)** following week

Always submit via Git

Solutions in a separate Git project



Eprog TA — HS 2026

janleonard115.github.io

Plan for today

- Theory Recap
 - Operator Precedence, Casting, Short circuiting, String Concatenations, Escape Characters
 - Exercises
- Kahoot
- VIS Exam Collection Guide
- Week 3 Exercise Rating
- Preview for next week

Theory

Evaluation – Arithmetic Operators

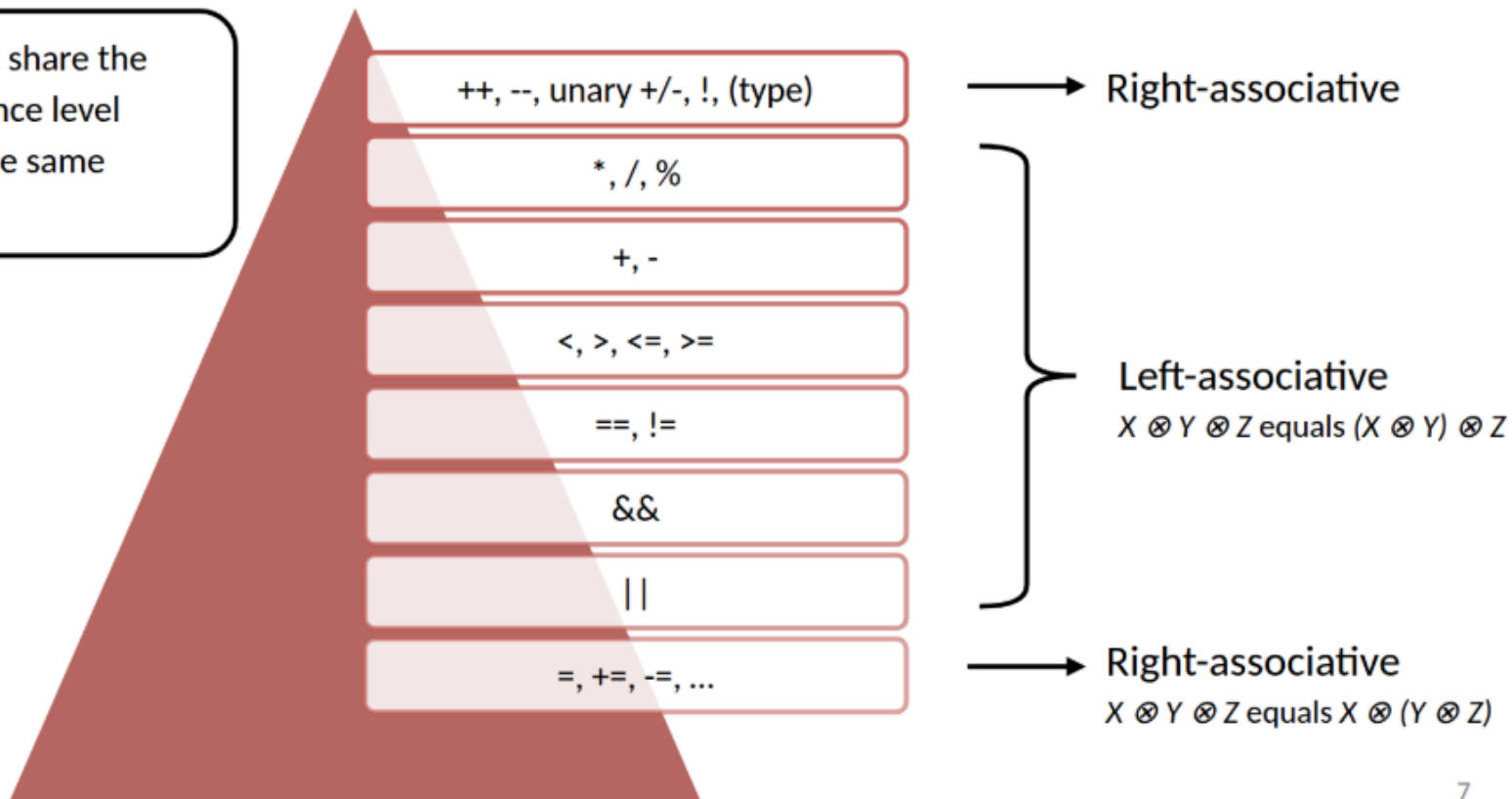
Operator	Beschreibung	Kurzbeispiel
+	Addition	<code>int antwort = 40 + 2;</code>
-	Subtraktion	<code>int antwort = 48 - 6;</code>
*	Multiplikation	<code>int antwort = 2 * 21;</code>
/	Division	<code>int antwort = 84 / 2;</code>
%	Teilerrest, Modulo-Operation, errechnet den Rest einer Division	<code>int antwort = 99 % 57;</code>
+	positives Vorzeichen	<code>int j = +3;</code>
-	negatives Vorzeichen	<code>int minusJ = -j;</code>

Precedence and Associativity

- **Expressions** can be read in different ways
 - $3 * 4 + 2$
- To get rid of ambiguity Java defines rules
- **Precedence** defines which operator binds more strongly
 - $(3 * 4) + 2$
- **Associativity** defines in which direction Operands get grouped if the Precedence is the same
 - $(10 - 4) - 3$

Operator Precedence – In Detail

Operators that share the same precedence level always have the same associativity.





Eprog TA — HS 2026

janleonard115.github.io

Associativity

The associativity of an operator determines how an operand is used

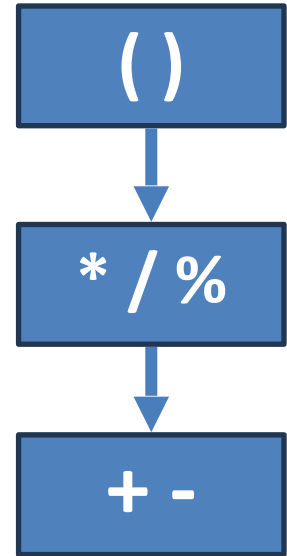
If an operator is $\otimes$

Left-associative $\rightarrow X \otimes Y \otimes Z$ equals $(X \otimes Y) \otimes Z$

Right-associative $\rightarrow X \otimes Y \otimes Z$ equals $X \otimes (Y \otimes Z)$

Evaluation – Order

- Operand is used by the operator with higher precedence
- If two operators have the same precedence, associativity decides
- If something else is desired: use parentheses!



String Concatenation & Escape Sequences

- If at least one **Operand** of + is a String
 - + becomes concatenation
 - Other operand is converted to a String
 - Same Precedence and Associativity rules as Plus apply
- Escape Sequences start with a \
 - \n, \t, \" \\ etc.
 - Get replaced by a single special character

Evaluation - Type

Same operands = Clear type

int + int = int

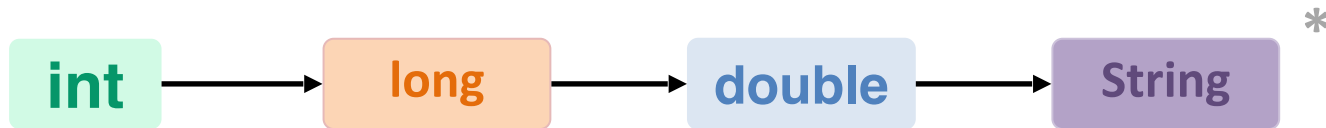
double + double = double

What if:

double + int = ?

Implicit casting

General rule:



Examples:

double + int = double

String + int = String

* This is an oversimplification. More types and exact casting will be covered later

```
System.out.println(8 - 3 * (5 - 2) + "=" + 17 % 5 + 9 / 2 * 2.0 + (int) 3.9 / 2);
```

Evaluating Expressions – Step by Step

Example: `10 - 2 * (1 + 3) + "x" + 7 / 2`

1 Resolve explicit parentheses

Innermost () first – use steps 2-3 inside it.

`(1 + 3) = 4 → 10 - 2 * 4 + "x" + 7 / 2`

2 Add the invisible parentheses

Higher in the pyramid → binds first.

Same level → follow the associativity.

`((10 - (2 * 4)) + "x") + (7 / 2)`

3 Evaluate one operator at a time

- int / int → int, cut off: `7 / 2 = 3`
- % → remainder: `8 % 3 = 2`
- int with double → double
- a String operand → + concatenates

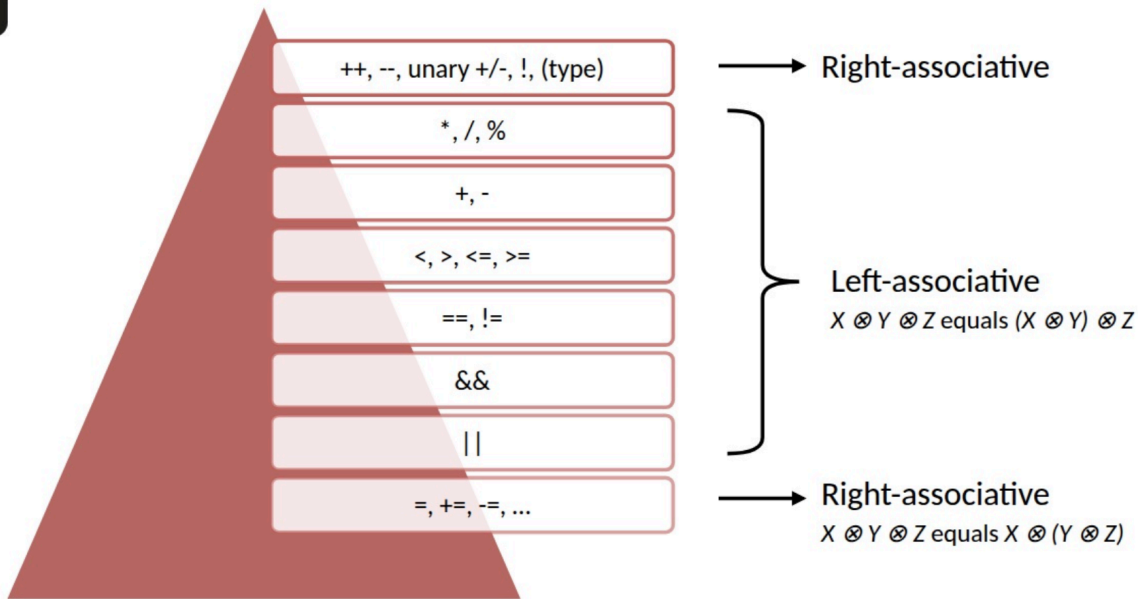
`2*4=8 → 10-8=2 → 2+"x"="2x"`
`→ 7/2=3 → "2x"+3="2x3"`

4 Write down the result

A double is always printed with a decimal point: `4.0`

Output: `2x3`

Note: This recipe works for expressions without side effects. With `++`, `--` or `=` inside an expression, remember that Java always evaluates operands left to right.



Operators that share the same precedence level always have the same associativity.

Precedence & associativity set the parentheses, **types** decide what each operator does.
When in doubt: **use parentheses!**

Evaluation – Examples

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(2/3+4.5/3);  
4     }  
5 }
```

Output: 1.5

Evaluation – Examples

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(8%3+2.5+4/3);  
4     }  
5 }
```

Output: 5.5

Evaluation – Casting

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println((double)5/2);  
4     }  
5 }
```

Output: 2.5

Evaluation – String

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Wir schreiben das Jahr "+2023/2*2.0);  
4     }  
5 }  
6
```

Output: Wir schreiben das Jahr 2022.0

Evaluation – Exam Example

()

* / %

+ -

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println(12/4*3+("X"+7%3)+18%5*2);  
4     }  
5 }
```

Output: 9X16

Evaluation – Exam Example

()

* / %

+ -

```
1
2 public class Main {
3     public static void main(String[] args) {
4         System.out.println("Wir schreiben das Jahr " + 2000 + 24);
5     }
6 }
```

Output: We are writing the year 200024

Exercise 1

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(9 / 3 * 2 + 1 + "" + 2 * 3 + (4 * 3) % 3);  
4      }  
5  }
```

760

Exercise 2

```
1  class MyClass{  
2      public static void main(String[] args){  
3          System.out.println(8 / 5 + 0.5 + 5 / 2 + (8 % 3) * 1.0);  
4      }  
5  }
```

5.5

Calculation - Exam Question

```
1. System.out.println(36 / (3 * 12    4));
```

```
//Output: 4
```

```
2. System.out.println(64 % (4    8 + 1) / 4);
```

```
//Output: 7
```

```
3. System.out.println(32    (2    6));
```

```
//Output -8
```

For each statement, determine the missing operators so that the statement produces the shown output. Possible operators are +, -, *, /, and %.

Calculation - Exam Question

```
1. System.out.println(36 / (3 * 12 / 4));
```

```
//Output: 4
```

```
2. System.out.println(64 % (4 * 8 + 1) / 4);
```

```
//Output: 7
```

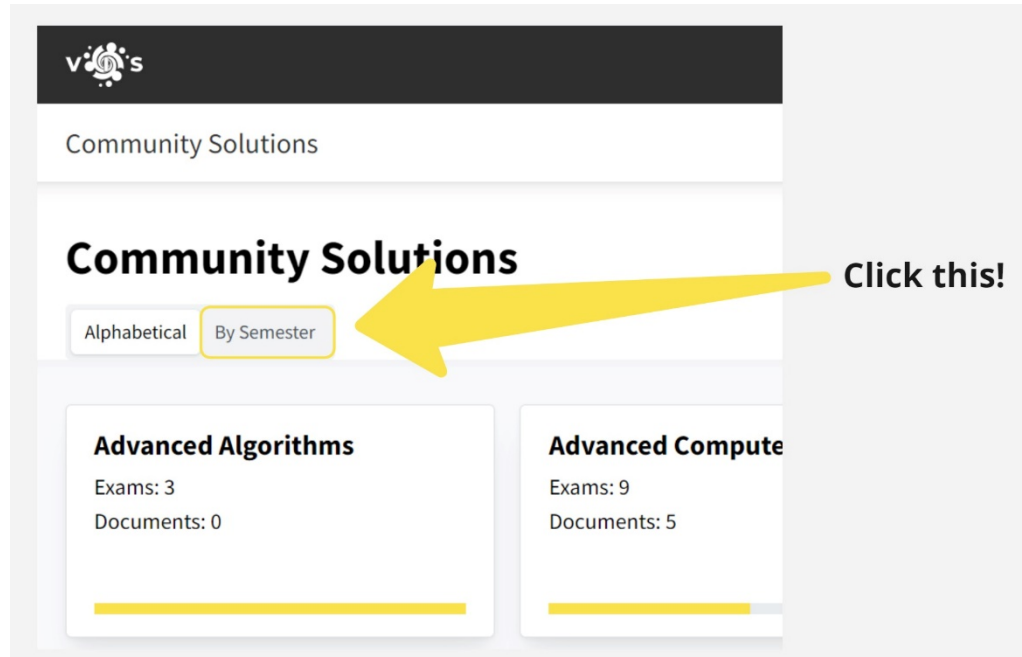
```
3. System.out.println(32 / (2 - 6));
```

```
//Output -8
```

For each statement, determine the missing operators so that the statement produces the shown output. Possible operators are +, -, *, /, and %.

VIS Exam Collection

Exams at: <https://exams.vis.ethz.ch>



Short-circuit evaluation

When computing `p && q` and `p || q`

Java first evaluates the left operand (`p`) and then the right one (`q`)
evaluation stops as soon as the result is fixed.

For `p && q`: if `p == false`, then `p && q == false`

For `p || q`: if `p == true`, then `p || q == true`

p	q	p && q
true	true	true
true	false	false
false	true	false
false	false	false

p	q	p q
true	true	true
true	false	true
false	true	true
false	false	false

In these cases, evaluating `q` is not needed!

Short Circuit Evaluation - Example

```
public class Main {  
    public static void main(String[] args) {  
        int x = 0;  
  
        if (x != 0 && 10 / x > 1) {  
            System.out.println("big");  
        } else {  
            System.out.println("x is 0 or too small");  
        }  
    }  
}
```

Kahoot

Review Exercise 2

Any further questions about IntelliJ or Git?

(Interactive)

Solution Exercise 2, Exercise 2

```
/*  
 * Author: Maximiliana Muster  
 * für Einführung in die Programmierung  
 */  
public class HelloProgrammer {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, my name is Max!");  
    }  
}
```

Solution Exercise 2, Exercise 4

1. Erstellen Sie eine Beschreibung `<geradezahl>`, die als legale Symbole alle geraden Zahlen (d.h. Zahlen, die ohne Rest durch 2 teilbar sind) zulässt. Beispiele sind +02, 4, 10, -20.

Solution Exercise 2, Exercise 4

2. Zeigen Sie in einer Tabelle, dass Ihre Beschreibung das Symbol “28” als gerade Zahl erkennt.

Solution Exercise 2, Exercise 4

3. Erstellen Sie eine Beschreibung `<x2ygemischt>`, die als legale Symbole genau jene Wörter zulässt, in denen für jedes "X" zwei "Y" als Paar auftreten. Beispiele sind `XYX`, `YYX`, `XYYYXX`, `XXYYYY`. Die Beschreibung schliesst den leeren String nicht aus.

Solution Exercise 2, Exercise 4

Die folgenden EBNF-Beschreibungen sind nicht äquivalent. Finden Sie ein *kürzestmögliches* Symbol, das von der einen Beschreibung als legal erkannt wird, aber nicht von der anderen. (Fangen Sie mit einfachen Kombinationen von A und B an.)

EBNF-Beschreibung: <beispiel1>

<beispiel1> $\Leftarrow$ [A][B]

EBNF-Beschreibung: <beispiel2>

<beispiel2> $\Leftarrow$ [A[B]]

Solution Exercise 2, Exercise 4

5. Erstellen Sie eine EBNF Beschreibung `<doppelt>`, die als legale Symbole genau jene Wörter zulässt, in denen die doppelte Anzahl "Y" nach einer Folge von "X" auftritt. Beispiele sind `XYX`, `XXYYYY`, usw. Die Beschreibung schliesst den leeren String nicht aus.

Preview Exercise 3

Exercise 1: Debugging in IntelliJ

1. Open the file `FollerVehler.java` in the folder `u03 → FollerVehler → src → main → java`.
2. Try to run the program. IntelliJ will show you the error messages.
3. First try to find the errors only by reading the code and thinking about how they could be fixed. (Tip: Pay attention to parentheses and braces, capitalization, semicolons, and method names.)
4. Fix the errors step by step until `FollerVehler.java` compiles and runs without errors. IntelliJ continues to show you the error messages and supports you in fixing the problems along the way.

When all errors have been fixed, the program should output the following:

```
Hello world
```

```
Gefällt Ihnen dieses Programm?
```

```
Ich habe es selbst geschrieben.
```

My Opinion: Good exercise, Do not just use IntelliJ as it will show you a lot

Exercise 1: Debugging in IntelliJ

- In Java, all Java files in a project are compiled.
- If even one Java file does not compile, no Java code can be executed in that project.
- Therefore, we use subprojects so that exercises 2-4 compile even if Exercise 1 does not compile.

Exercise 2: Swiss Flag

1. Create a new Java file "SwissFlag.java". To do this, right-click the *src* folder and select *New* → *Java Class*. Enter the name of the class ("SwissFlag.java") and confirm with the Enter key. Then select *Add* if you are asked whether you want to add the file to git.
2. First create an empty `main` method, as you have seen in other programs.
3. Extend the program so that it prints the Swiss flag in the console. The flag could look roughly as follows, but you may also design a larger or nicer version:

Divide the program into several methods that are called by the `main` method. This ensures that there are fewer repetitions of code fragments, which makes changing the program much easier.

```

|                                     |
|                                     |
|          ++++++                    |
|          ++++++                    |
|          ++++++                    |
|      ++++++ ++++++ ++++++          |
|      ++++++ ++++++ ++++++          |
|      ++++++ ++++++ ++++++          |
|          ++++++                    |
|          ++++++                    |
|          ++++++                    |
|                                     |
|-----|-----|

```

My Opinion: Not as important but if you do it then use clean code.

Exercise 3: Input and Randomness

In this task, you work with Java input and output and get to know the classes `Scanner` and `Random` more closely. The classes `Scanner` and `Random` must be imported. This can be done with `import java.util.Scanner` and `import java.util.Random`, as shown in the lecture.

1. Write a program “Adder.java” that reads in two integers and prints their sum. You should use the `Scanner` class for this, as shown in the lecture. The program should ask for the first number:

Geben Sie Zahl 1 ein:

then for the second number:

Geben Sie Zahl 2 ein:

and finally, if you enter for example “25” and “1999”, output the following:

25 + 1999 = 2024

You can assume that only integers are entered. Test the program with different numbers.

My Opinion: Good Exercise to practice Input/Output

2. Write a program `Wuerfel.java` that simulates a die. A positive integer `N` should be read in; it represents the number of sides of the die. The usual die has 6 sides, but dice with 12, 16, 20 (see Figure 1) and more sides also exist. Each side carries a different number, ranging from 1 to `N` (including `N`).

The program should simulate the roll by using the `Random` class. One possible run of the program could look as follows:

Wie viele Seiten hat Ihr Würfel?

The user enters a number, e.g. 20; after that, the die is rolled:

Es wurde eine 17 gewürfelt!



3. Write a program `ChatBot.java` that simulates interaction with an AI chatbot. The program should greet the user, ask for their name and age, and then reveal the lucky number. The lucky number should be chosen randomly by using the `Random` class. One possible run of the program could look as follows:

Guten Tag! Ich bin ChatBot, der beste Chatbot, den es gibt! Wie heissen Sie?

A name is entered, e.g. Alice; after that, the chatbot responds to this input:

Sehr erfreut Alice! Wie alt sind Sie?

An age is entered, e.g. 23; after that, the chatbot responds to this input:

Mittels dieser Informationen habe ich Ihre Glückszahl gefunden! Die Glückszahl lautet 42.

My Opinion: Most important exercise. Solve it without help and think about how you get the first and second digit out of an int.

Exercise 4: Calculations

1. Complete “SharedDigit.java”. In the main method, two int variables a and b are declared and initialized with a value between 10 and 99 (inclusive). The program should assign a certain value to an int variable r. If a and b have a digit in common, then r is assigned the common digit (if a and b have both digits in common, then any one digit may be assigned). If there is no common digit, then -1 should be assigned. You do not need a loop for this program.

Examples:

- If a: 34 and b: 53, then r: 3
- If a: 10 and b: 22, then r: -1
- If a: 66 and b: 66, then r: 6
- If a: 34 and b: 34, then r: 3 or 4

Test your solution with a equal to 34 and b equal to 43. What does your program return?

2. Complete "SumPattern.java". In the main method, three int variables a, b, and c are declared and initialized with arbitrary values. If the sum of two of the variables gives the third, suppose $a + c == b$, then the method should output "Moeglich. $a + c == b$ " (where the values for a, b, and c must be inserted). If this is not the case, then the method should output "Unmoeglich."

Examples:

- If a: 4, b: 10, c: 6, then "Moeglich. $4 + 6 == 10$ " or "Moeglich. $6 + 4 == 10$ " is output.
 - If a: 2, b: 12, c: 0, then "Unmoeglich." is output.
3. Complete "AbsoluteMax.java". In the main method, three int variables a, b, and c are declared and initialized with arbitrary values. The program should assign to an int variable r the largest absolute value of a, b, and c.

Exercise 5: EBNF

Note: This EBNF task is also available as a [Moodle quiz](#). In some parts, the Moodle quiz contains additional exercises and is therefore also helpful for students who intend to work only on the paper version.

In this task, you again create various EBNF descriptions. Save these as usual in the text file “EBNF.txt”, which is located in the “u03” → “resources” folder. You can edit the file directly in IntelliJ.

1. Create a description `<pyramid>` which allows, as legal symbols, exactly those words that consist of a sequence of strictly increasing digits followed by a sequence of strictly decreasing digits. Examples are: 14, 121, 1221, 1341.
You may assume that the empty word is also allowed. (As a challenge, you can try to exclude the empty word.)

Important to keep practicing EBNF and get used to the exam format

Exercise 5: EBNF

2. Create a description `<digitSum>` which allows, as legal symbols, exactly those natural numbers whose digit sum is an even number. (As a challenge, you can try to exclude the empty word.)
3. Create a description `<xyz>` which allows exactly all words consisting of X, Y, and Z in which every X has at least one Y in the subword to its left and to its right. Examples are: Z, YXY, YXXY, ZYXYY.
4. Create a description `<term>` which allows, as legal symbols, exactly all well-formed arithmetic terms consisting of positive integers, variables (x, y, z), addition, and parentheses. Parenthesized terms must contain at least one addition. Examples are: $1 + 4$, $(1 + 4)$, $1 + (3 + 4)$, $(1 + 1) + x + 5$.



EPROG Questions Week 3

docs.google.com

Quick Tips

- If you need git push -f, Something went wrong
- Technical Problems during the timed bonus or in the exam: what to do

Thank you very much for your attention

Have a great week!